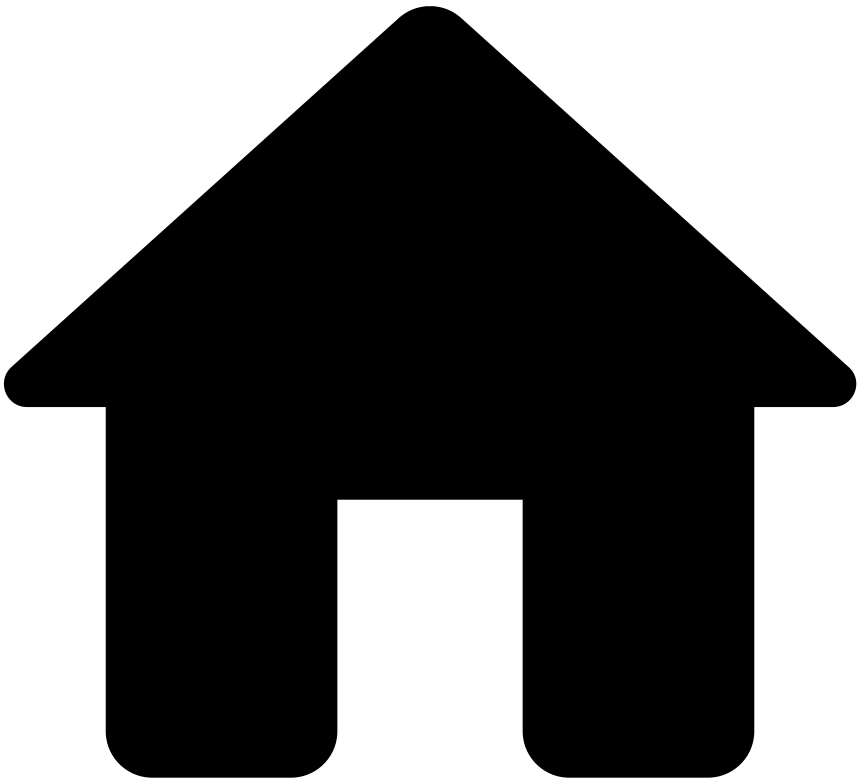


On this page

Lists

Was this page helpful?Â 💬

A list stores a set of items. Lists are available for rules to check the same pattern against a large number of values. E.g., a list of credit card numbers, a list of merchants, etc.

In the process of fraud detection is very common to define blacklists for blocking transactions from certain cards, or even define transactions with common parameters that should be up for review (triggering an alarm every time a transaction occurs). Lists allow you to group and manage these values so that you can use them efficiently with Rules.

When you define a list, the first thing you need to consider is which list type is best suited to your use case. Lists are essentially made of items, typically hundreds of them, where information will be stored. Depending on the items you must choose between one of the following:

- **Simple list**: When only one piece of information is needed. Represents a list containing only numbers or strings (commonly used for blacklists/whitelists).
- **Complex list**: When you need to store complex values, for example tuples. Each item is identified by a key and that key has a predefined set of fields. For example, the key of an item may be *country* and its fields can be a set of *safe_merchants* for that country.

  Complex lists allow you to define rules that first check for the existence of the key and then verify conditions over the other fields. For example, when a transaction comes into the system, lookup if the country of the transaction is in the list of risky countries. If so, check if the transactions were done on "safe merchants" for that country.

List Configurationâ□□

Imagine you want to create a list to store patterns used by potentially dangerous transactions. A rule can then verify if a transaction matches any dangerous pattern stored in the list and help reduce fraudulent transactions.

As an example, we will define the dangerous payments pattern as a country, a list of MCCs and a list of terminal authentication methods. The next steps describe how such a list can be created.

When configuring a list, pay special attention to the following parameters:

- **Name**: Choose a representative name as it will be used as the internal list name in RDL. This internal name can't be changed and has some differences to the list name. For example, `"DangerousPayments"` will generate an internal name of `"dangerous_payments".`
- **Tags**: Describe briefly the purpose of the list. Can be used in API calls to filter existing lists. For example, add the `suspicious` keyword to all the lists that contain dangerous patterns.
- **Key Label**: Lists are implemented as a key-value store. The key can store multiple types of information ranging from card numbers, to MCCs or country names.
- **Key Type**: Pulse supports two key types: **String** and **Number**. Consider the internal implementation of the data structure and access capabilities of lists to understand why limits the available key types.
- **Fields**: Defines the schema, or the internal structure, of items in a complex list.

High cardinality fields

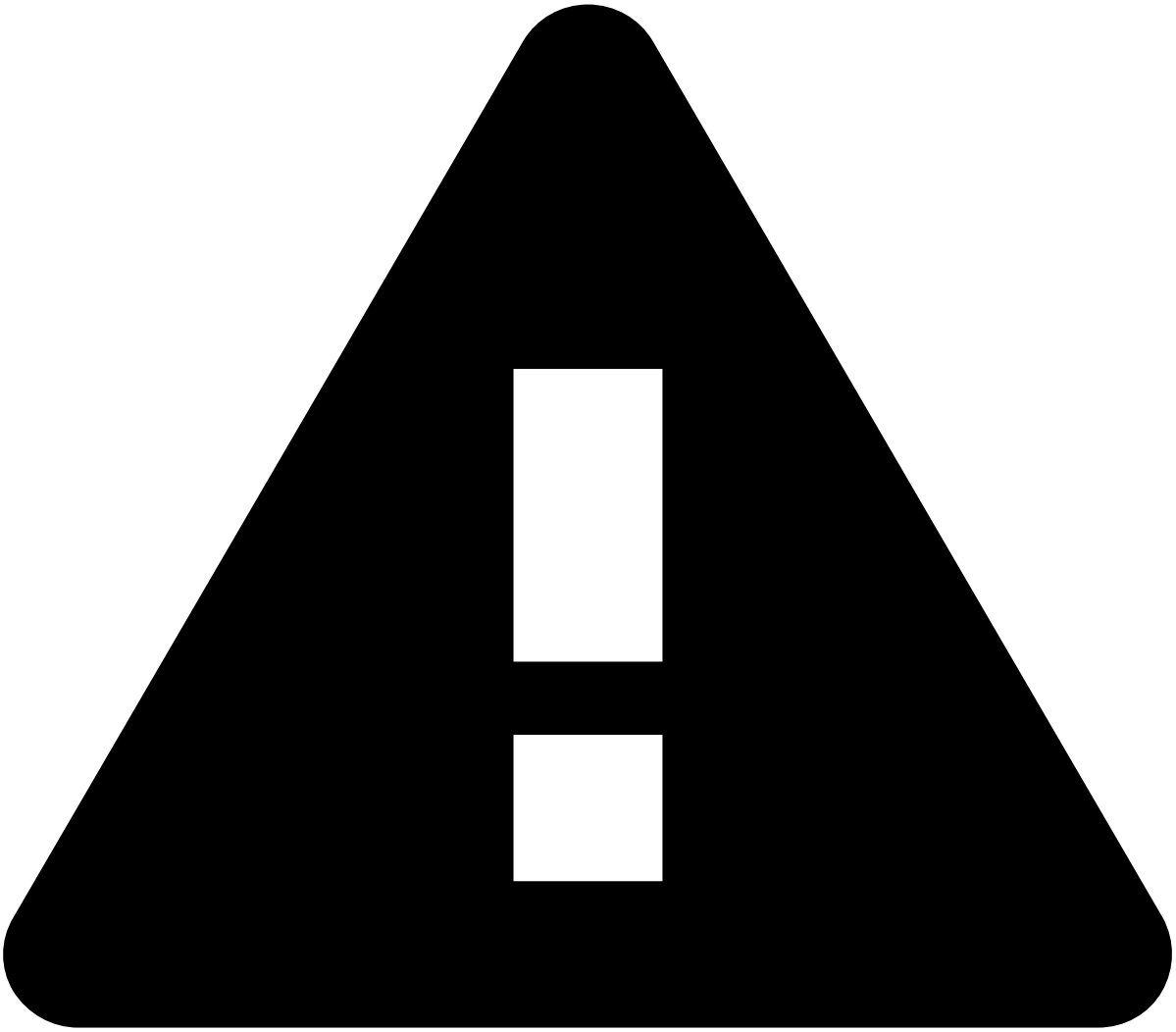
Each list item will be stored in memory from the moment Feedzai Pulse starts.

This has a direct impact on the consumed memory, thus having the potential to cause problems with fields that have high value cardinality.

List Itemsâ

After you have created a List you need to add items to it. In the case of a `DangerousPayments` list, the items are fraudulent patterns that represent suspicious transactions.

You can load entire lists, with a large number of entries, in bulk. Feedzai Pulse accepts a CSV file with the items' information and handles the addition process for each item transparently.

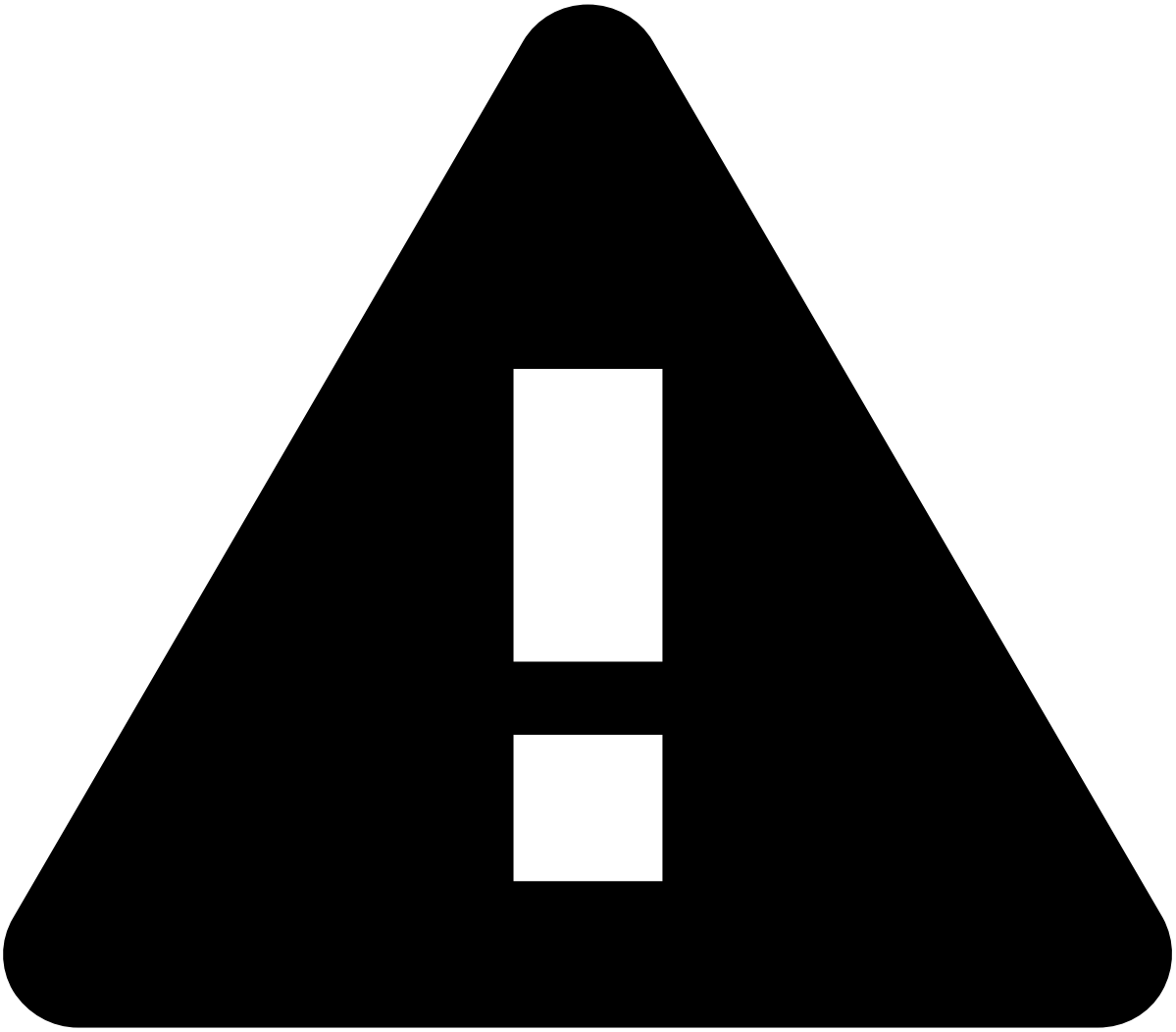


Upload Limit Recommendation

To ensure reliable processing and avoid straining system resources, it is recommended that each uploaded list file contains at most 40,000 items.

Note that this figure is a general guideline; the actual threshold for optimal performance can vary depending on the specific contracted resource sizing. If your list is larger than that, split it into multiple files before uploading.

Timespan support in items is an essential feature when creating watchlists. These lists allow you to, for example, add a card number that should be under watch during a given period of time after having suspicious activity. Lists support the following three timespan types:



Key uniqueness

The conjunction of the item `key` + `timespan` must be unique. In other words, Pulse will not add the card `0000-0000-0000-0000` to a List if it is already there with an overlapping timespan. For multi-tenant lists, the tenant unique id is also part of the unique key: `key + timespan + tenant.`

Note that the timespans are interpreted according to the workflow clock. See Time, schedules and timezones for more information.

Multi-Tenancyâ□□

Lists in Pulse also support the multi-tenancy concept. A list can be of the type **Global** if the list is aimed to be used in a context that is not Multi-Tenanted or of the types **By Tenant** and **By Tenant Group** if the list is aimed to be used in a multi-tenancy project.

In a list of the type **By Tenant**, each Tenant will be able to see and manage a subset of all the items without impacting other Tenants.

To further detail how to create lists for multi-tenancy, visit Creating Lists and Adding Items.

Tokenized Listsâ□□

Once you have set up the Tokenizer, you will also be able tokenize lists. For example, black lists can be sometimes composed of Credit Card numbers, which in order to fulfil PCI DSS compliance, must be encrypted. Therefore, lists can be tokenized by specifying it during its creation. After defining the list as tokenized, you can see how the sensitive information has been encrypted.

For instance, when downloading the list of items, the CSV file no longer shows clear text PAN numbers, but tokens instead. All CSVs also show a column that indicates whether the key of the item is tokenized. In downloads, that column displays a **True** value if the list is encrypted, or **False** if it is not. When uploading, if set as **True**, it works as selecting the **Encrypted** checkbox in the item creation screen.

Reviewing List Changesâ□□

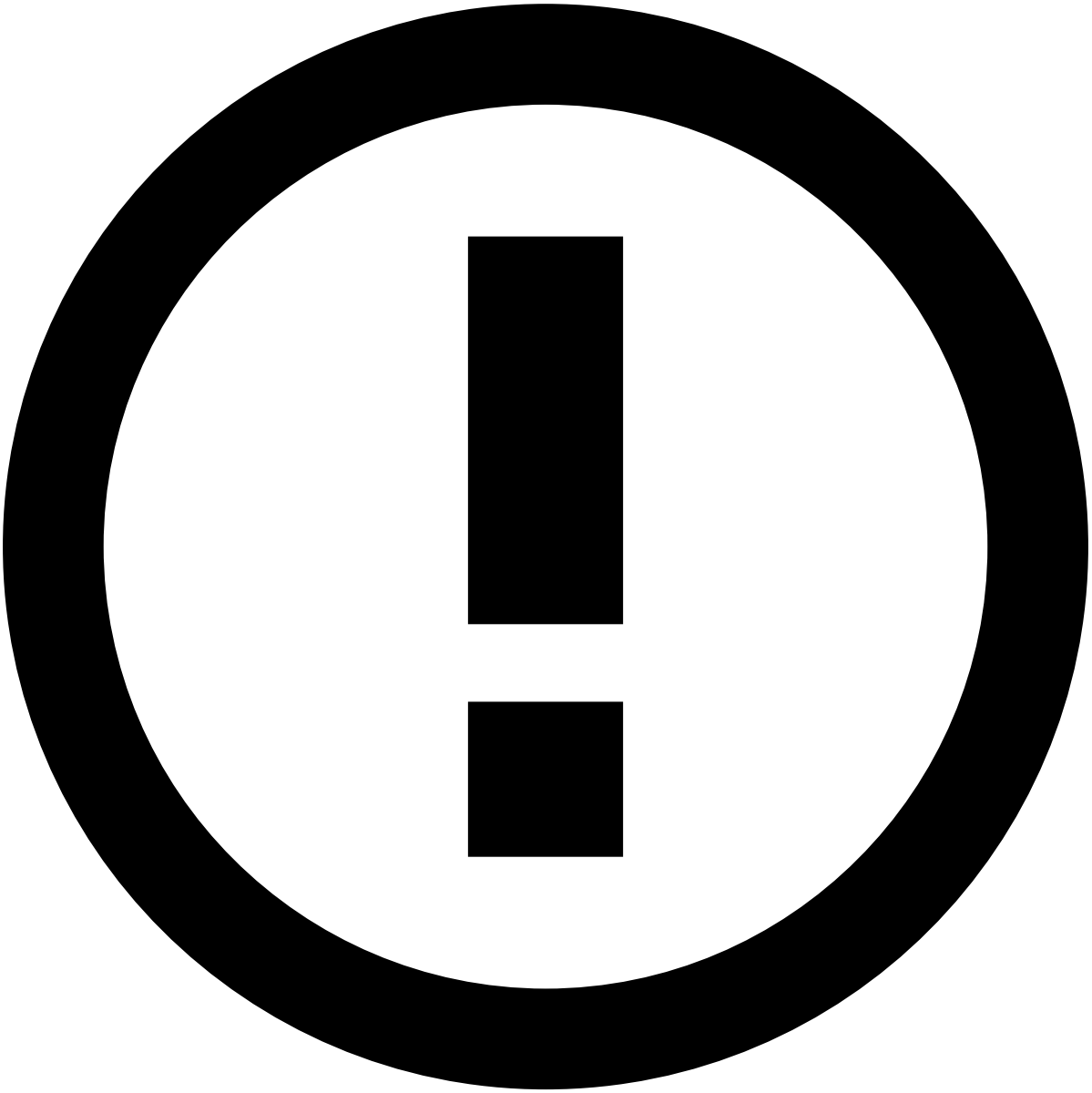
Pulse supports a "4-eye check" review functionality for list changes, which places edits made to lists into a review queue, and only applies the changes if they are approved by a second person.

To learn more about this functionality, see Reviewing List Changes - Four Eyes Check.

Auditingâ□□

The purpose of auditing is to store the logs of changes (creation, edition and deletion) made to items in the lists. These audit logs display, among other data, when and who made changes to the items, as well as the corresponding new field values.

The audit entries are only accessible via Pulse REST API through a query by item key.



Pulse Database

Auditing is supported only by the relational Pulse Database (PDB).

Note that the audit entries are not visible in Pulse User Interface (UI); they are stored and available for other applications to fetch them.